AMTCS: *IoT project on* Automating Measurements Using a Temperature Controller and Spectrometer: Initially Simulated

(client: Tyndall National Institute, UCC)

Marina Zubialevich

Final project

Higher Diploma in Computer Science, SETU

Cork, 2026

Introduction

- Motivation

  Temperature-resolved photoluminescence is a powerful characterisation technique in solid state physics

- Problem

  Time consuming, prone to user errors, requires constant monitoring and repetitive user interventions, induces excessive user exposure to laser radiation, etc.

- Solution

  Repetitive and time consuming **tasks can be automated** reducing staff time and laser radiation hazard risks, making the technique more error-proof and efficient

AMTCS System Architecture

Complications:

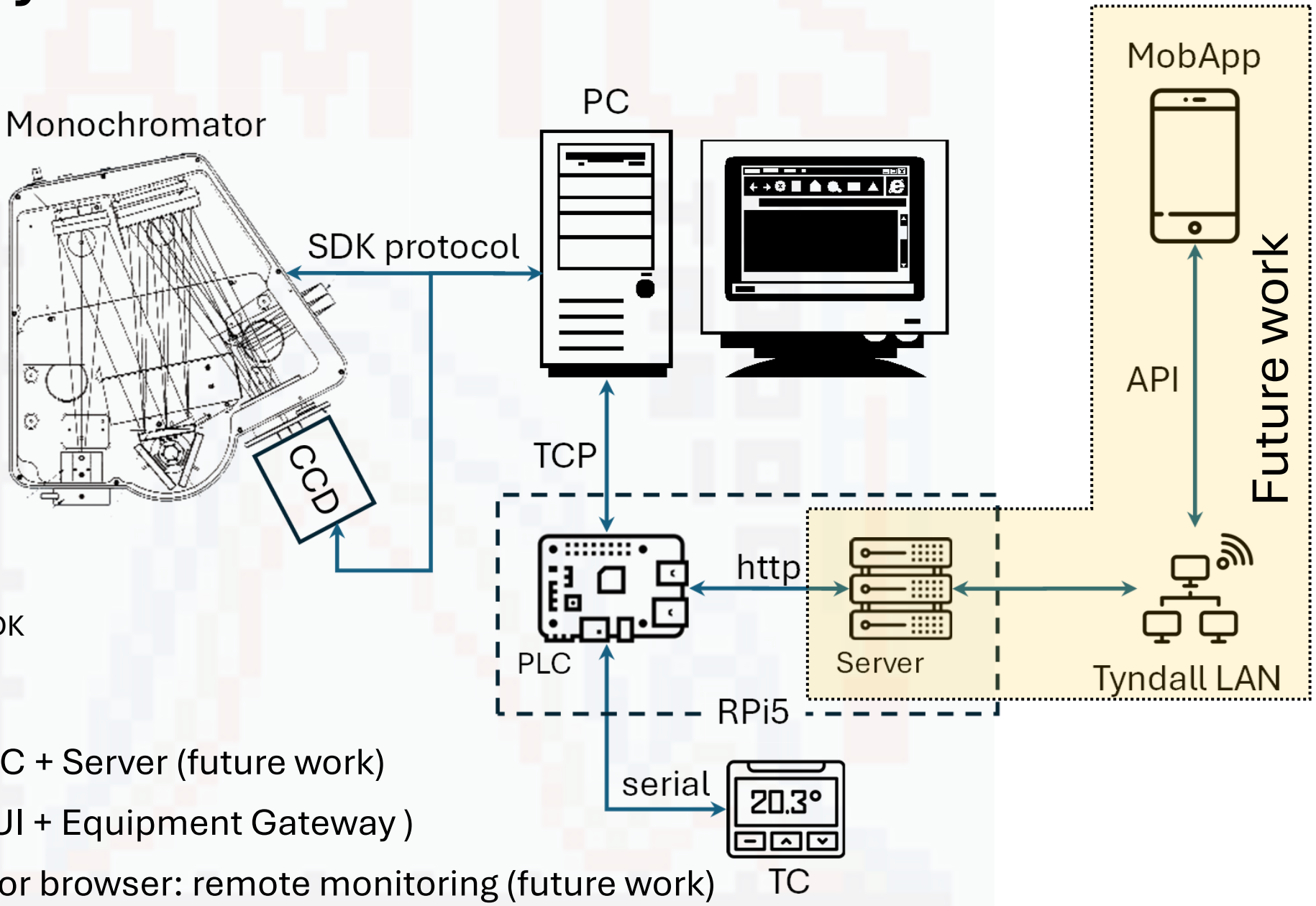
- Equipment from different manufacturers
- Legacy SDK (VBA6-era)
- Legacy OS (W7)

Languages:

- C++ for UI App
- Python for PLC

Communication protocols:

- PLC ↔ UI App: TCP
- PLC ↔ TC: serial
- UI App ↔ Spectrometer: SDK



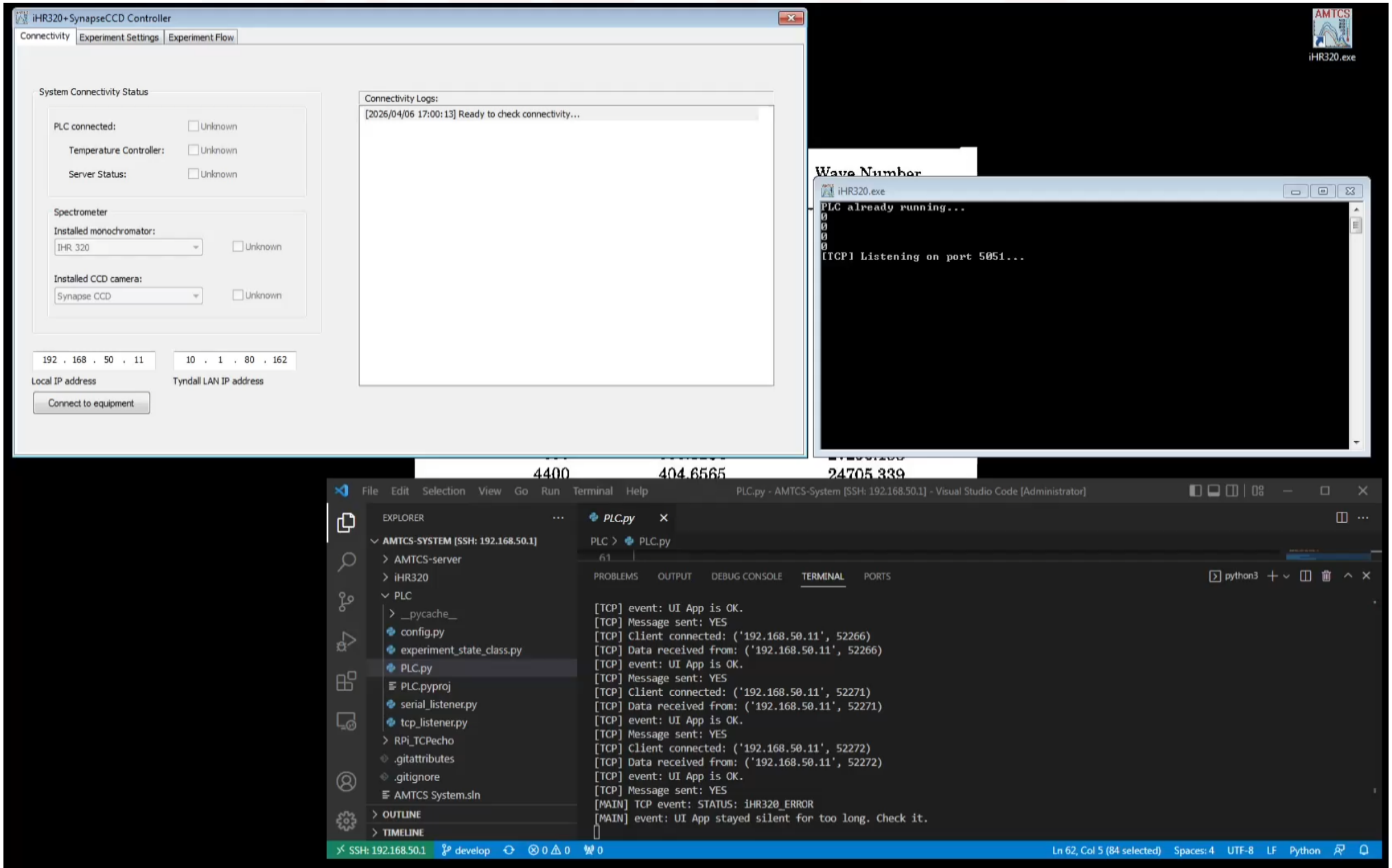
- Raspberry Pi: PLC + Server (future work)
- Windows 7 PC (UI + Equipment Gateway)
- Mobile app and/or browser: remote monitoring (future work)

AI usage

AI was used as support tool:

- For C++ coding/debugging (to mitigate the steep learning curve)
- For reasoning of architectural solutions
- For final report writing (English, style)

Short demonstration



Methodology

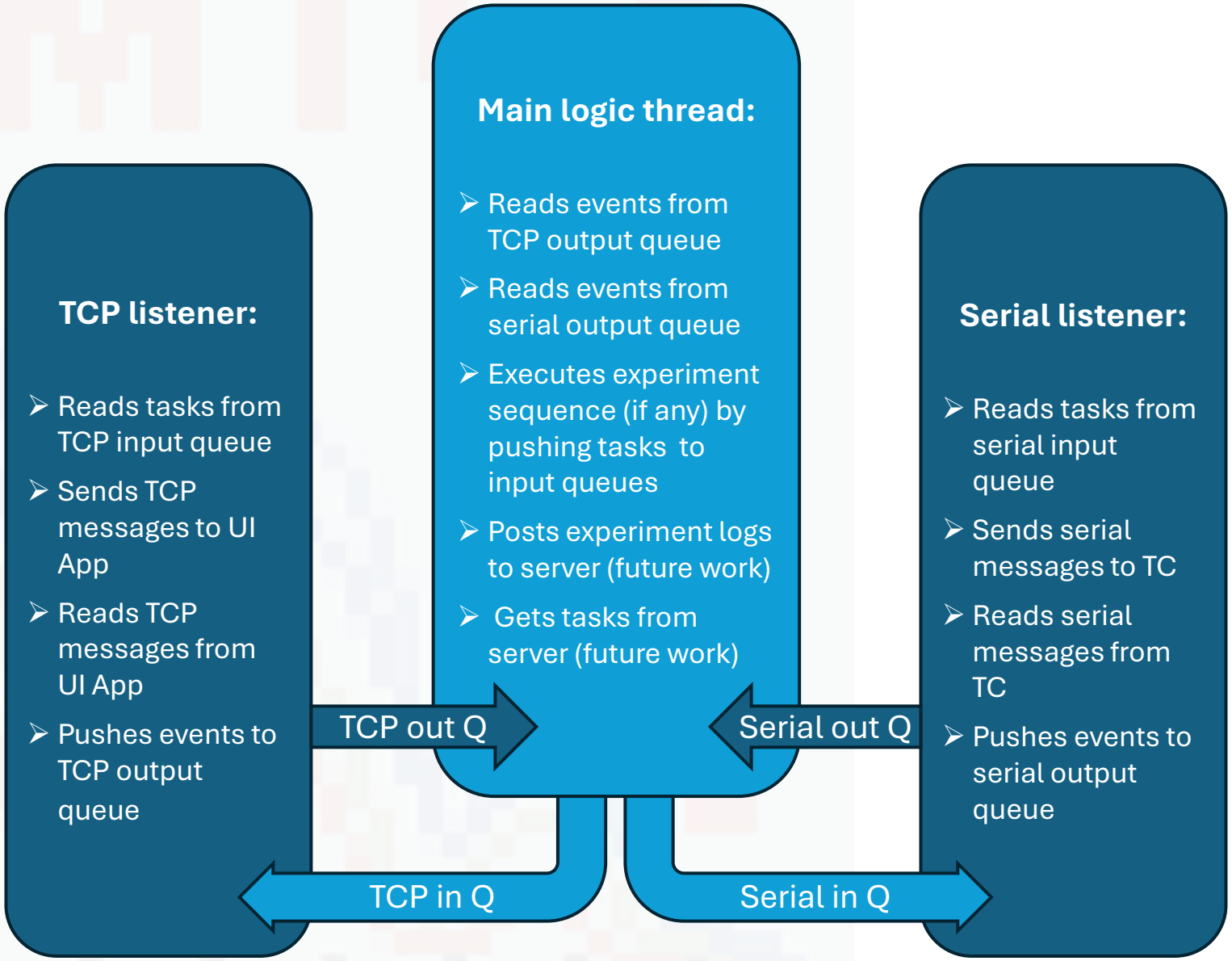
- Agile
  - Trello
  - Sprints
  - Weekly meetings with Tyndall mentor
- Simulation (VM environment)
- Actual hardware use (real lab environment)
- Git Flow for version control

Sprints as planned

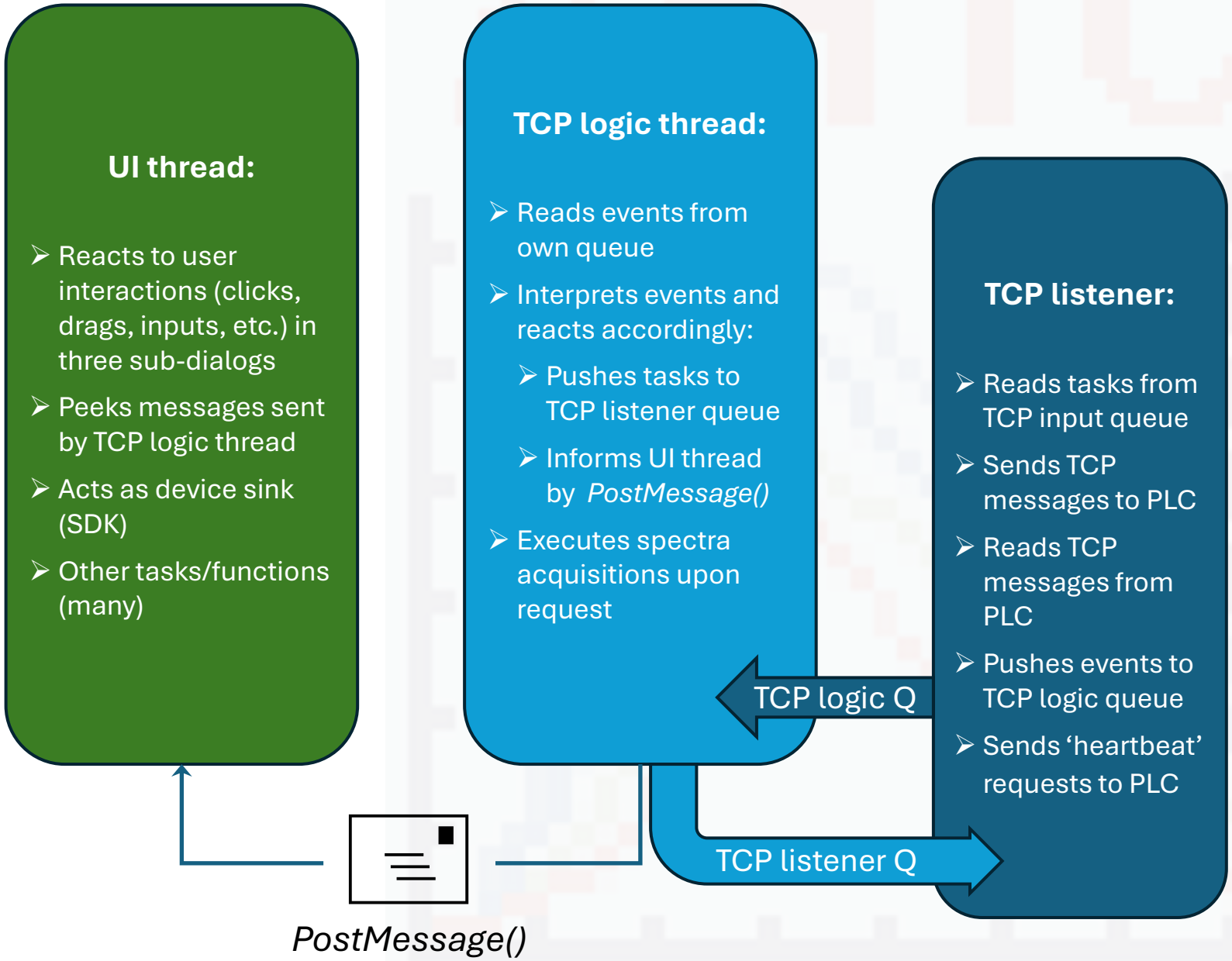
1. RAMP
2. System architecture + communication
3. Thermocontroller integration
4. SDK-based equipment integration (spectrometer)
5. Orchestration, logging
6. Testing, refinement

PLC structure

- Three-thread approach for the PLC process:
  - Main logic
  - TCP listener
  - Serial listener
- Thread-proof queues for event/task exchange between threads
- Clear task separation between each thread

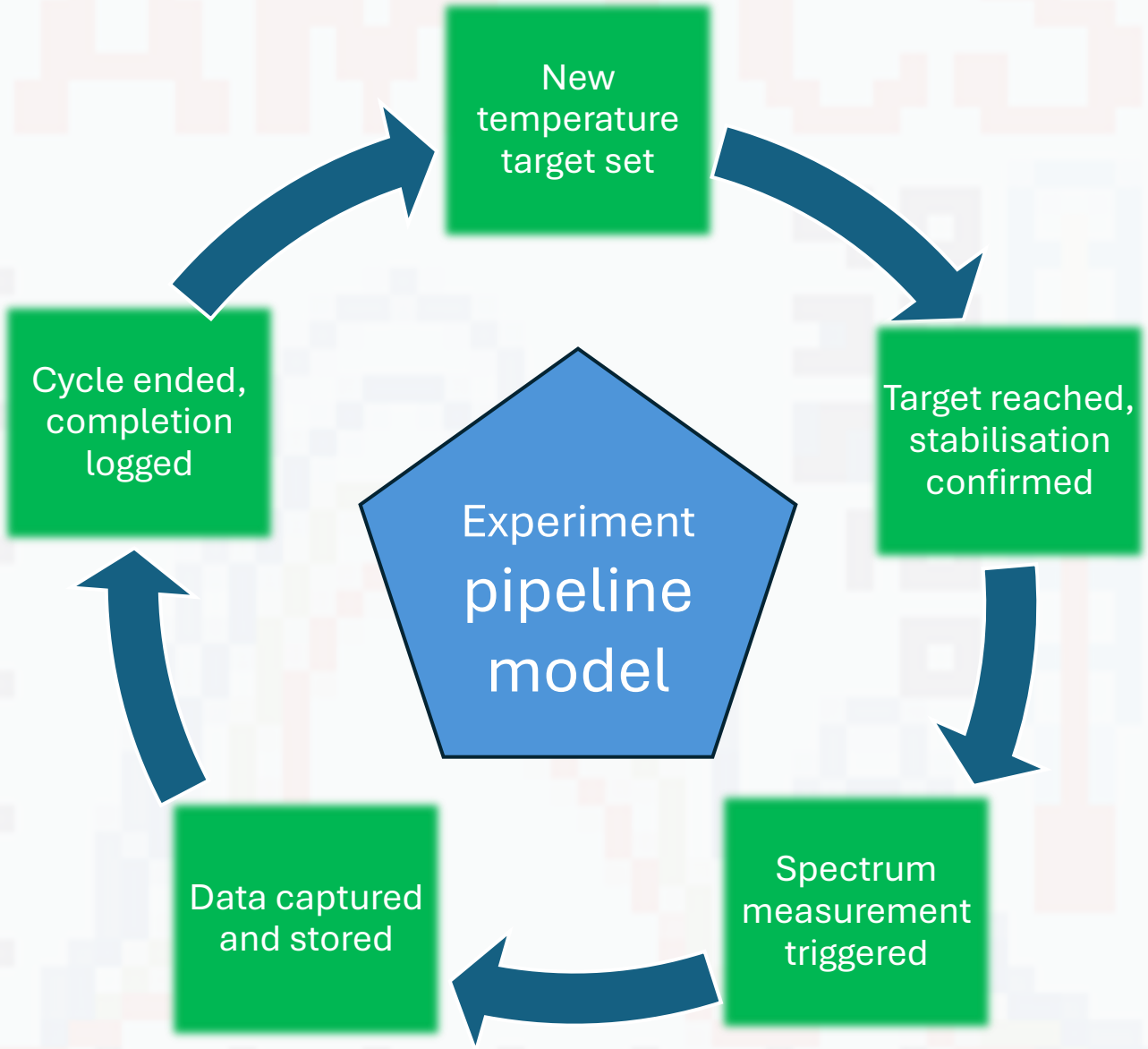


UI App structure



- MFC App enriched by two additional threads:
  - Main UI (MFC)
  - TCP logic
  - TCP listener
- Thread-proof queues for event/task exchange between TCP threads
- Unidirectional communication from TCP logic thread to UI thread

Desired experiment sequence



Experiment preparation and orchestration

Connection/Initialisation (UI App + PLC)

Experiment setup (UI App)

Validation/Experiment State (UI App → PLC)

Experiment cycle/steps

TEMPERATURE			SPECTRUM		
T.WAITING	T.REQUESTED	T.COMPLETED	S.WAITING	S.REQUESTED	S.COMPLETED
• Requests TC for a new T target • Reads current target from TC • If both coincide, sets T.REQUESTED	• Periodically reads current T from TC • If within ±0.5°C of the target => stabilisation • If long enough in stabilisation, sets T.COMPLETED	• Logs target has been reached • Sets S.WAITING	• Requests UI App for spectrum measurement • Reads from UI App • If confirmed, sets S.REQUESTED	• Reports awaiting for spectrum acquisition confirmation • Reads from UI App • If 'DONE' received, sets S.COMPLETED	• Logs spectrum has been acquired • Increments experimental cycle

Next temperature/experiment cycle

Conclusions

All originally planned MVP features have been implemented:

- ✓ Serial communication between the PLC and TC
- ✓ Temperature stabilisation detection
- ✓ SDK-based spectrometer measurements (triggered by PLC)
- ✓ Experiment sequencing logic (PLC-style orchestration)
- ✓ Automated data capture, saving and logging
- ✓ Local experiment status (UI interface)

Learnings:

- !! Planning is essential (splitting to sprints very helpful)
- !! Write down/document things, do not rely on memory

Future works

	Status
○ Initially planned:	
• Detection/correction of cosmic ray artefacts	Realised
• Autoexposure feature	Realised
• Server for logging (API provider)	Started
• Browser/Mobile App client (API consumer)	Not yet started
○ Identified in the development stage:	
• Experiment recovery (e.g. after a crash)	Realised

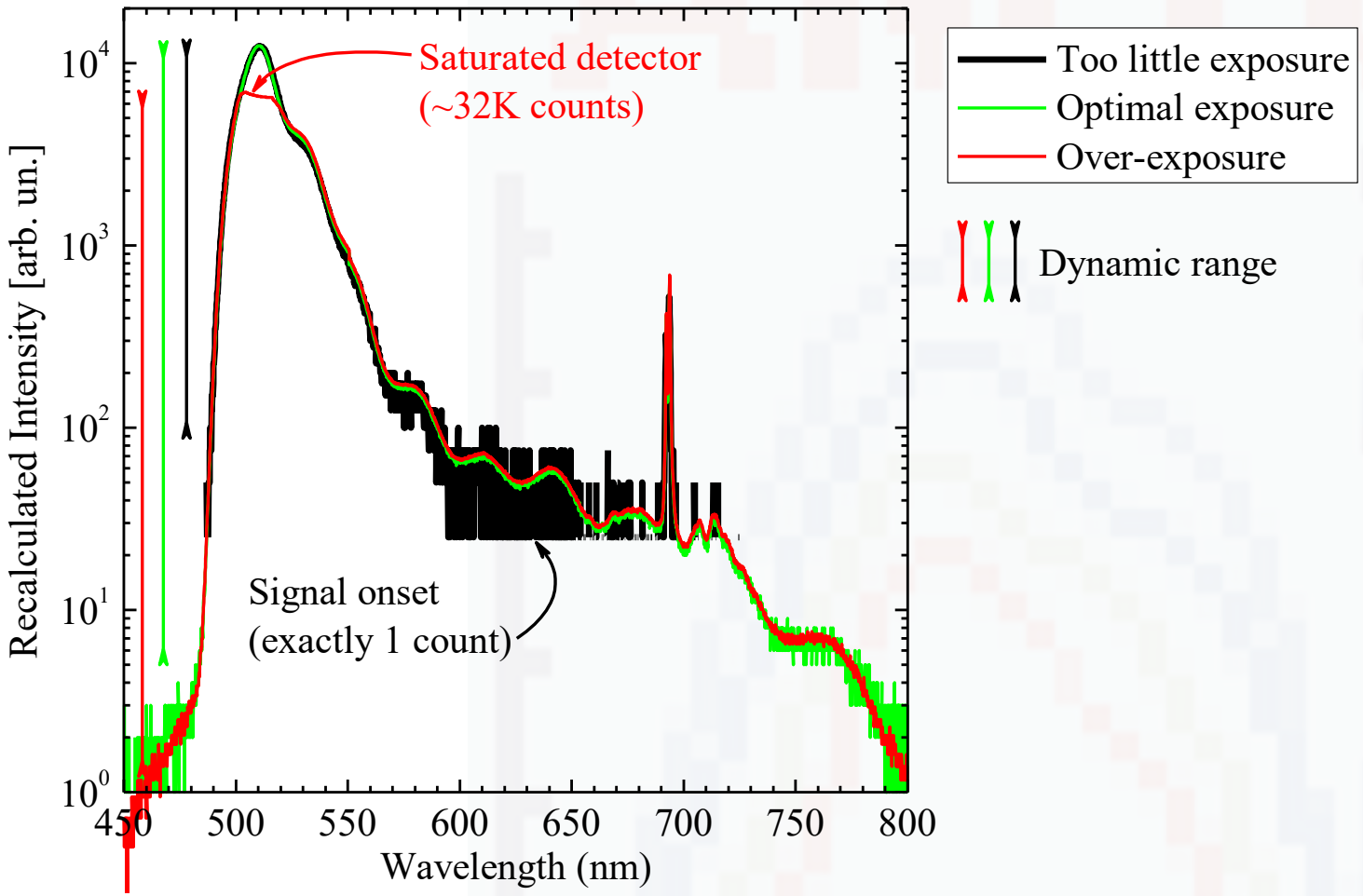
Cosmic ray removal

- ‘Cosmic rays’ artefacts caused by spontaneous bit switch e.g. 0100001011011 10

  Switched bit Actual signal

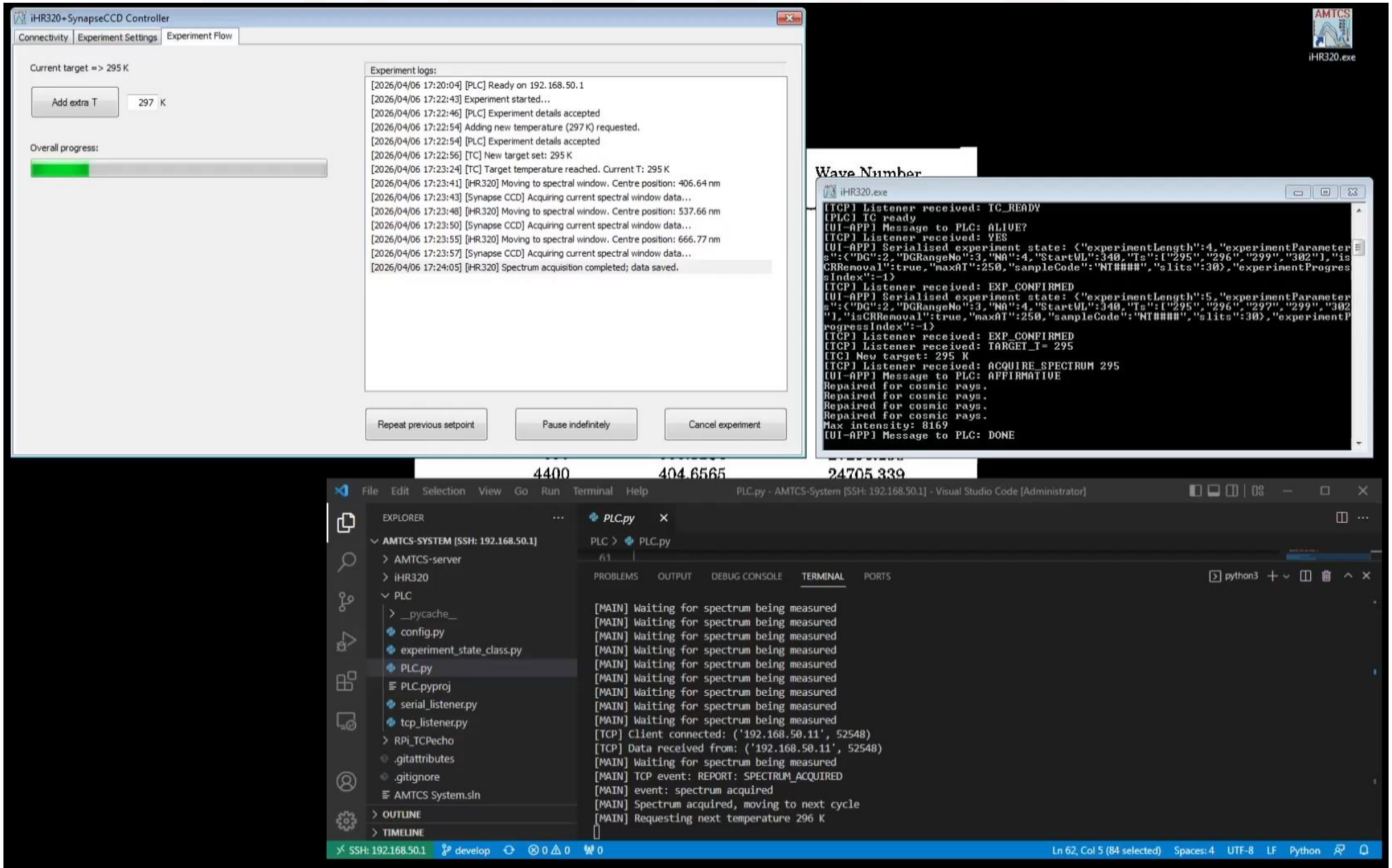
- In practice, such a pixel reading will strongly differ in intensity from the rest of acquisitions
- Algorithm suggested by researchers:
  - Average of all acquisitions is calculated: $\overline{y} = \frac{1}{n}\sum_{i=0}^{n-1} y_i$
  - Absolute relative deviations $\frac{\Delta y_i}{\overline{y}} = \left|\frac{y_i - \overline{y}}{\overline{y}}\right|$ are calculated and compared to 20% (threshold)
  - Highest absolute relative deviation is considered to be a ‘cosmic ray’ artefact
  - Such a pixel data is discarded and replaced with the average of the rest (for this reason minimum three acquisitions are required!)
  - The subroutine is recursed until no defective pixel is detected
- The feature was essential for autoexposure functionality

Autoexposure

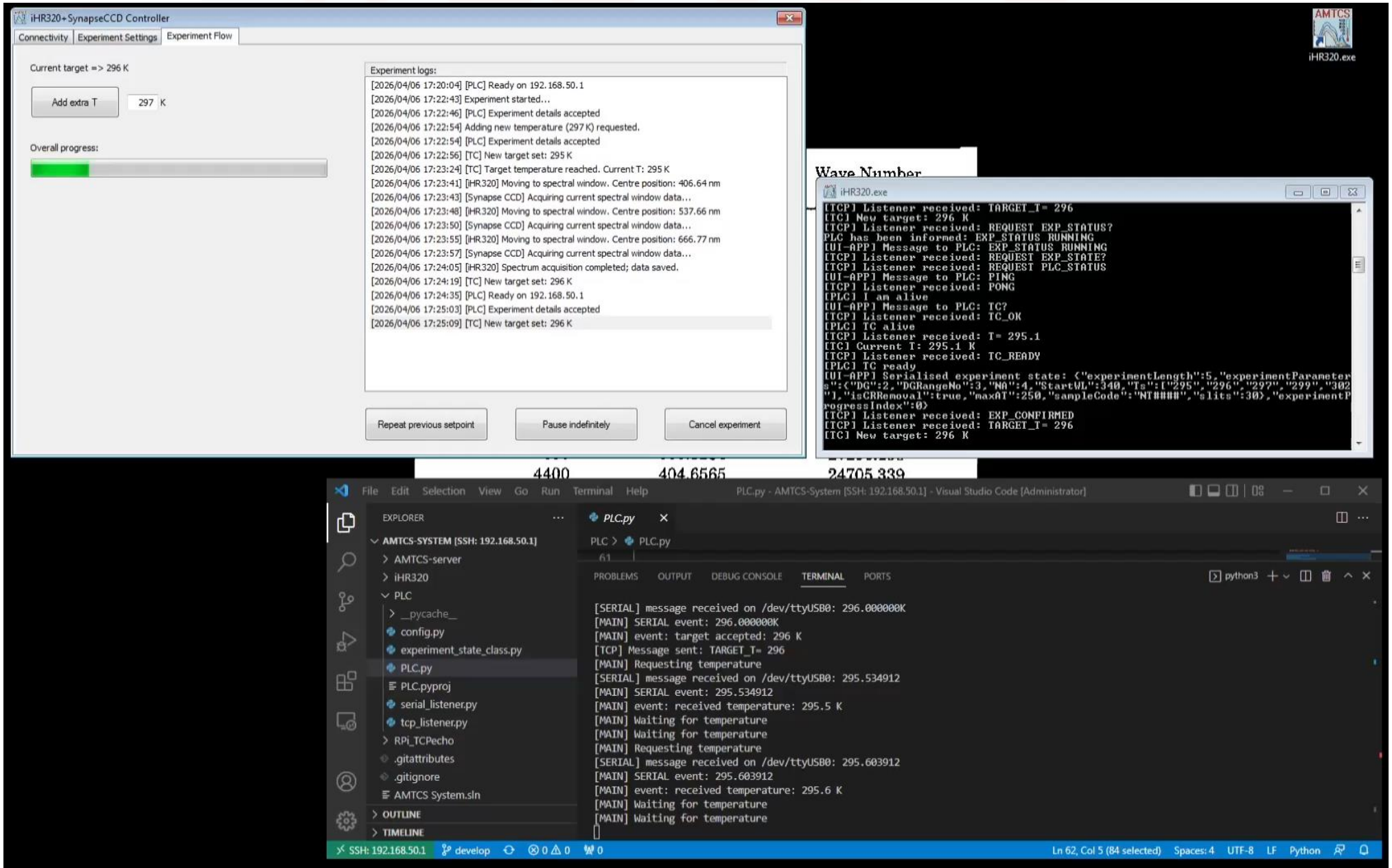


- In spectrometry, dynamic range is an essential parameter of quality of measurement
- It shows for how many orders of magnitude measured signal spans
- High dynamic range allows to see signals significantly different in intensity
- Dynamic range is defined by spectrum maximum intensity
- Too much light can saturate detector (bad)
- **Target here was to get max signal to a level between 6.25 and 50% of full scale of CCD camera (2000-16000 counts).**

Experiment Recovery (PLC crashed)



Experiment Recovery (UI App crashed)



Thank you for your attention!